recursion tree

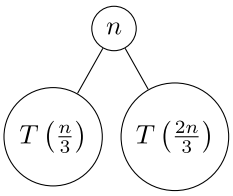
mahmood

February 12, 2023

Example 0.0.1. consider the following

$$T(n) = T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + n$$

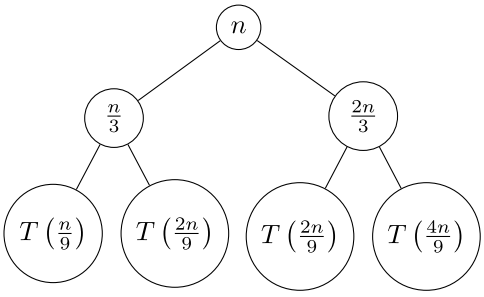
a note to keep in mind is that the sum of all the nodes of the tree must always be equal to $T(n)$
with that in mind, the first step would be:



if we sum all the nodes we can see that indeed $T(n) = T\left(\frac{2n}{3}\right) + T\left(\frac{n}{3}\right) + n$ for the next step we need to write $T\left(\frac{2n}{3}\right)$ and $T\left(\frac{n}{3}\right)$ in terms of time complexity for smaller values of n so we can know what the next row of nodes would be

$$T\left(\frac{2n}{3}\right) = T\left(\frac{2 \cdot \frac{2n}{3}}{3}\right) + T\left(\frac{\frac{2n}{3}}{3}\right) + \frac{2n}{3} = T\left(\frac{4n}{9}\right) + T\left(\frac{2n}{9}\right) + \frac{2n}{3}$$
$$T\left(\frac{n}{3}\right) = T\left(\frac{2 \cdot \frac{n}{3}}{3}\right) + T\left(\frac{\frac{n}{3}}{3}\right) + \frac{n}{3} = T\left(\frac{2n}{9}\right) + T\left(\frac{n}{9}\right) + \frac{n}{3}$$

according to this, the tree with the new nodes would be:

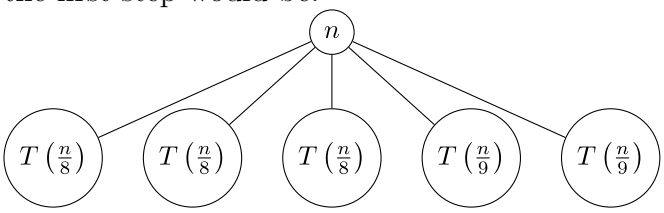


we define a **full row** as a row that is full of nodes, in the previous tree the rows $1, 2, 3$ are full, we might notice that the sum of all the nodes of a full row is n
let y be the last full row
because the leftmost node is always the smallest in its row, for a row to be full the leftmost node has to be greater or equal to 1 ($\frac{n}{3^y} \geq 1$ in this case)
if $\frac{n}{3^y} \geq 1$ is true then the leftmost node in the row $y+1$ whose value is $\frac{n}{3^y}$ appears in the tree (because nodes with values greater or equal to 1 do appear in the tree), and so the row $y+1$ is full, which results in a contradiction because y is the last full row
therefore it must be that $\frac{n}{3^y} < 1 \Rightarrow n < 3^y \Rightarrow y > \log_3 n$
therefore the tree contains atleast $\log_3 n$ full rows, and because the sum of every full row is n we get:
$T(n) =$ sum of all nodes $=$ sum of nodes in full rows $+$ sum of nodes in unfull rows $\geq$ sum of nodes in full rows $\geq n \cdot \log_3 n = \Theta(n \log n)$
and therefore $T(n) = \Omega(n \log n)$
let x be the last row in the tree, the value of the rightmost node in row x would be $\left(\frac{2}{3}\right)^{x-1} \cdot n$
if this value is less than 1 then this node doesnt appear in the tree and all the other nodes in row x that have a smaller value also dont exist which gives us a contradiction because x is the last row in the tree
therefore we get $\left(\frac{2}{3}\right)^{x-1} n \geq 1 \Rightarrow n \geq \left(\frac{3}{2}\right)^{x-1} \Rightarrow \log_{3/2} n \geq x - 1 \Rightarrow x \leq 1 + \log_{3/2} n$
and therefore in the tree there is at most $1 + \log_{3/2} n$ rows, and since the sum of every row in the tree is n we get:
$T(n) =$ sum of all nodes $=$ sum of all nodes in all rows $\leq n \cdot$ number of rows in the tree $\leq n \cdot (1 + \log_{3/2} n) = \Theta(n \log n)$
and therefore $T(n) = O(n \log n)$
in summery, we proved that $T(n) = \Theta(n \log n)$

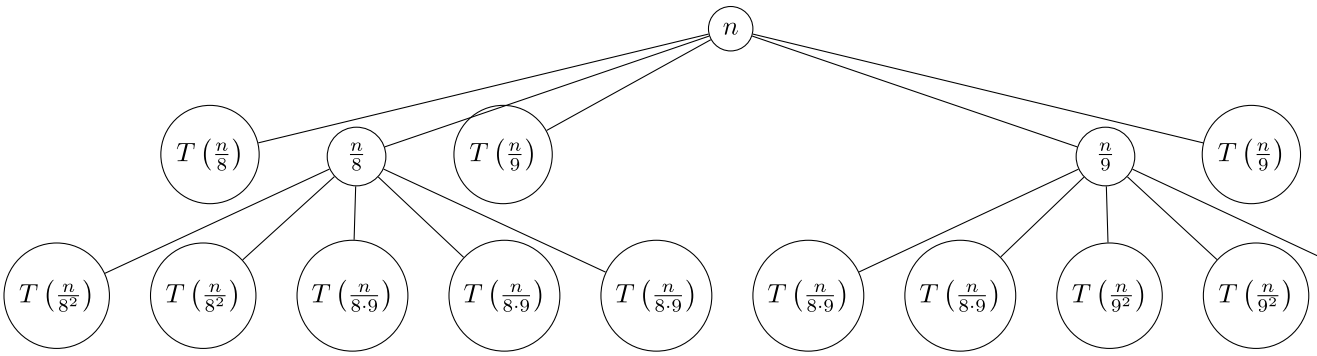
Example 0.0.2. consider the following

$$T(n) = 2T\left(\frac{n}{8}\right) + 3T\left(\frac{n}{9}\right) + n$$

we cant determine Θ using the , but we can using a recursion tree
the first step would be:



$$T\left(\frac{n}{8}\right) = 2T\left(\frac{n}{8^2}\right) + 3T\left(\frac{n}{8 \cdot 9}\right) + \frac{n}{8}$$
$$T\left(\frac{n}{9}\right) = 2T\left(\frac{n}{8 \cdot 9}\right) + 3T\left(\frac{n}{9^2}\right) + \frac{n}{9}$$



the sum of the nodes in level 1 is n, in level 2 its $n\left(\frac{2}{8} + \frac{3}{9}\right)$, in level 3 its $n\left(\frac{2}{8} + \frac{3}{9}\right)^2$, and so in level i it is $n\left(\frac{2}{8} + \frac{3}{9}\right)^{i-1}$
we denote by x the last level

$$
\begin{aligned}
T(n) &= \text{sum of all nodes} \\
&= \text{sum of all nodes in full rows} \\
&\leq n + n\left(\frac{2}{8}+\frac{3}{9}\right) + n\left(\frac{2}{8}+\frac{3}{9}\right)^2 + \cdots + n\left(\frac{2}{8}+\frac{3}{9}\right)^{x-1} \\
&\leq n\left(\left(\frac{2}{8}+\frac{3}{9}\right) + \left(\frac{2}{8}+\frac{3}{9}\right)^2 + \cdots + \left(\frac{2}{8}+\frac{3}{9}\right)^{x-1}\right) \\
&= n \cdot \frac{1}{1-\left(\frac{2}{8}+\frac{3}{9}\right)} \\
&= \Theta(n)
\end{aligned}
$$

note the sum was found using geometric progression formula
and so we have shown that $T(n) = O(n)$

$$T(n) = \text{sum of all nodes in the tree} \geq \text{root} = n = \Theta(n)$$

and so we have shown that $T(n) = \Omega(n)$
and so $T(n) = \Theta(n)$

note

Example 0.0.3. this need some correction as the sum of the nodes in the tree isnt exactly 2^{x-1}

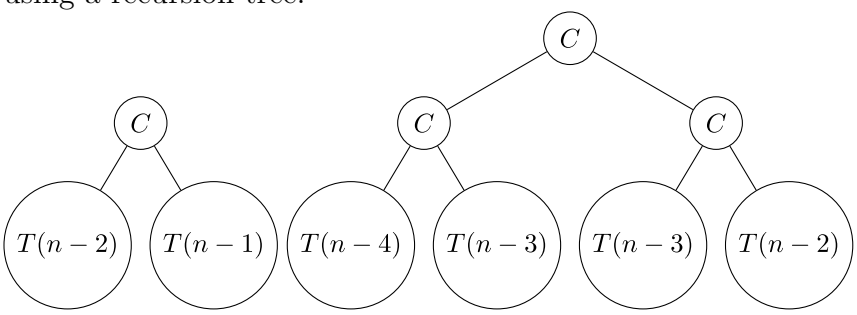
```
best_sum(A, i) {
  if i > size(A)
    return 0
  return max(best_sum(A, i+1), A[i] + best_sum(A, i+2))
}
```

$$
\begin{aligned}
T(n) &= T(n-1) + T(n-2) + C \\
&\leq 2T(n-1) + C \\
&\leq 2(2T(n-2) + C) + C = 4T(n-2) + 3C \\
&\leq 4(2T(n-3) + C) + 3C = 8T(n-3) + 7C \\
&\vdots \\
&\leq 2^{n-1}T(n-(n-1)) + \left(2^{n-1} - 1\right) C \\
&\leq 2^{n-1}T(1) + \left(2^{n-1} - 1\right) C \\
&\leq 2^n = \Theta(2^n)
\end{aligned}
$$

we proved $T(n) = O(2^n)$, now we try showing $T(n) = \Omega(2^n)$

$$
\begin{aligned}
T(n) &= T(n-1) + T(n-2) + C \\
&\geq 2T(n-2) + C \\
&\geq 2(2T(n-4) + C) + C = 4T(n-4) + 3C \\
&\geq 4(2T(n-6) + C) + 3C = 8T(n-6) + 7C \\
&\vdots \\
&\geq 2^{n/2}T(n-(n-2)) + \left(2^{n/2} - 1\right) C \\
&\geq 2^{n/2}T(2) + \left(2^{n/2} - 1\right) C \\
&\geq 2^{n/2} = \Theta\left(2^{n/2}\right)
\end{aligned}
$$

and so we showed that $T(n) = \Omega\left(2^{n/2}\right)$, which means there isnt a tight bound on the time complexity of this function
using a recursion tree:



$$
\begin{aligned}
T(n) &= \text{sum of all nodes} \\
&= \text{sum of all nodes in all levels} \\
&\leq 2^{x-1} \\
&\leq 2^{x} = \Theta(2^{x})
\end{aligned}
$$

we denote by y the last level with a full row, for this row to exist, $n-2(y-1) \geq 0 \implies n+2 \geq 2y \implies \frac{n+2}{2} \geq y$, so we have $\frac{n+2}{2}$ full rows

$$
\begin{aligned}
T(n) &= \text{sum of all nodes} \\
&\geq \text{sum of full rows} \\
&\geq \sum_{i=1}^{\frac{n+2}{2}} 2^{i-1} \\
&\geq 2^{\frac{1}{2}n+1} - 1 \\
&\geq \frac{2^{n/2}}{2} - 1
\end{aligned}
$$

the division by 2 doesnt affect big omega so $T(n) = \Omega\left(2^{n/2}\right)$